
SOC ANALYST PROJECT

Azure Cloud Honeypot

Deployment, Monitoring, and Attack Analysis

Microsoft Azure · Microsoft Sentinel · Log Analytics · KQL

Prepared by : Nakedi Tumelo Peta

Environment : Microsoft Azure (East US 2)

Contents

Introduction	3
Honeypot configuration and deployment	3
1. Azure subscription	3
2. Resource group	3
3. Virtual network	4
4. Virtual machine deployment	4
5. VM access and management	5
Monitoring and logging.....	6
6. Local logging.....	6
7. Forwarding logs to Azure Sentinel	6
Attack simulation and analysis.....	6
8. Simulation	6
9. Data visualization and querying.....	6
Visualization	8
Conclusion.....	8

Introduction

This project set up a honeypot in Microsoft Azure to capture and study how attackers behave against an exposed Windows host. A honeypot is a system left deliberately reachable so that malicious traffic is drawn to it, which gives a defender a safe place to watch real attack attempts. The build sends its security logs to Microsoft Sentinel, where failed logins can be queried, enriched with location data, and plotted on a map.

The sections below walk through the Azure environment, the virtual machine, the logging pipeline into Sentinel, and the analysis of the login attempts that the honeypot recorded.

Honeypot configuration and deployment

1. Azure subscription

The resources run on a free Azure subscription created with a personal email account. Azure normally asks for a credit card at sign-up. A Cyber Range subscription removes that step, which is the route used here.

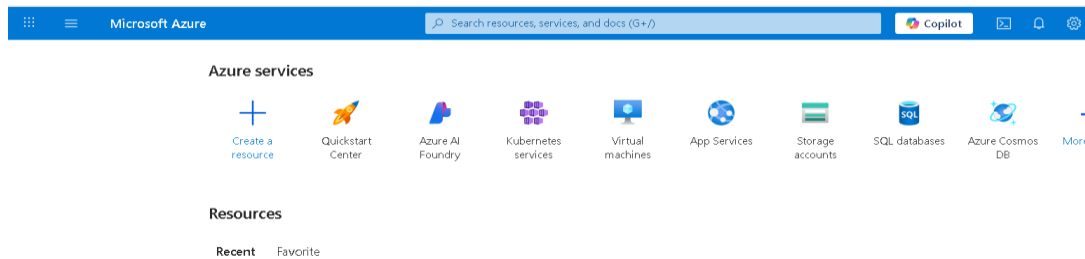


Figure 1. Azure portal home with the core services used in the build.

2. Resource group

A resource group named RG-SOC-Lab holds every resource for the lab so they can be managed and deleted as one unit. The deployment region is East US 2.

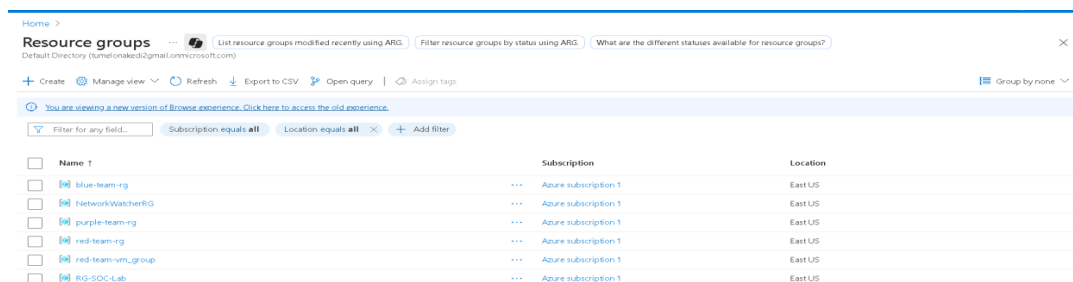


Figure 2. Resource groups in the subscription, including RG-SOC-Lab.

3. Virtual network

A virtual network (VNet) sits in the same region, East US 2, with its own address range of 10.0.0.0/16. The VNet keeps the virtual machine on its own segment, the same way a home router separates devices behind it.

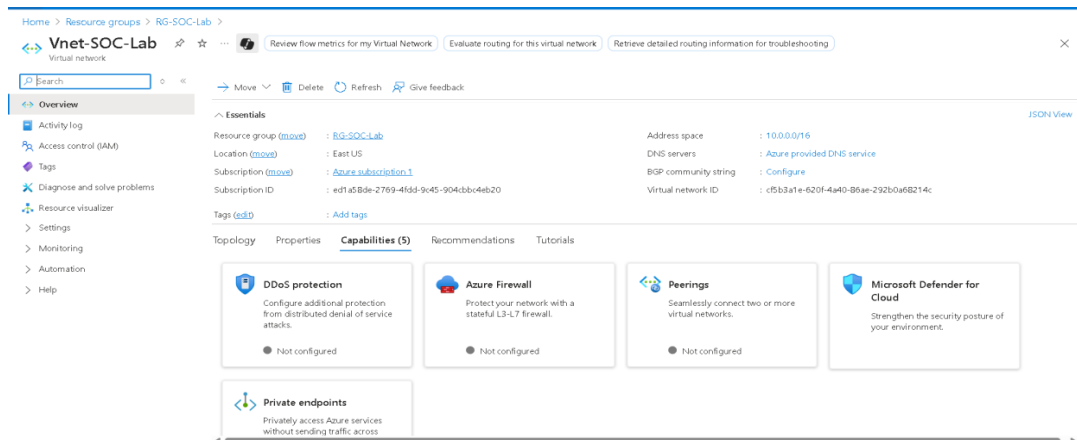


Figure 3. Vnet-SOC-Lab overview and capabilities.

4. Virtual machine deployment

A Windows 10 Pro virtual machine was deployed into the VNet with the following settings:

- **Name:** CORP-NET-EAST-1
- **Size:** Standard D2s v3 (2 vCPUs, 8 GiB), chosen to keep cost down
- **Access:** a public IP for external connectivity
- **Authentication:** username and password set during setup
- **Network:** attached to the VNet and its default subnet

The network security group (NSG) was set to allow inbound Remote Desktop Protocol (RDP), and other ports were left open on purpose, so the host looks vulnerable and attracts traffic. Pinging the public IP confirmed the machine was reachable from outside the network.

```

Microsoft Windows [Version 10.0.22631.5768]
(c) Microsoft Corporation. All rights reserved.

C:\Users\tumza>ping 40.76.251.214

Pinging 40.76.251.214 with 32 bytes of data:
Reply from 40.76.251.214: bytes=32 time=256ms TTL=105
Reply from 40.76.251.214: bytes=32 time=244ms TTL=105
Reply from 40.76.251.214: bytes=32 time=705ms TTL=105
Reply from 40.76.251.214: bytes=32 time=246ms TTL=105

Ping statistics for 40.76.251.214:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    approximate round trip times in milli-seconds:
        Minimum = 244ms, Maximum = 705ms, Average = 362ms

C:\Users\tumza>

```

Figure 4. Ping to the VM public IP (40.76.251.214), four replies, no loss.

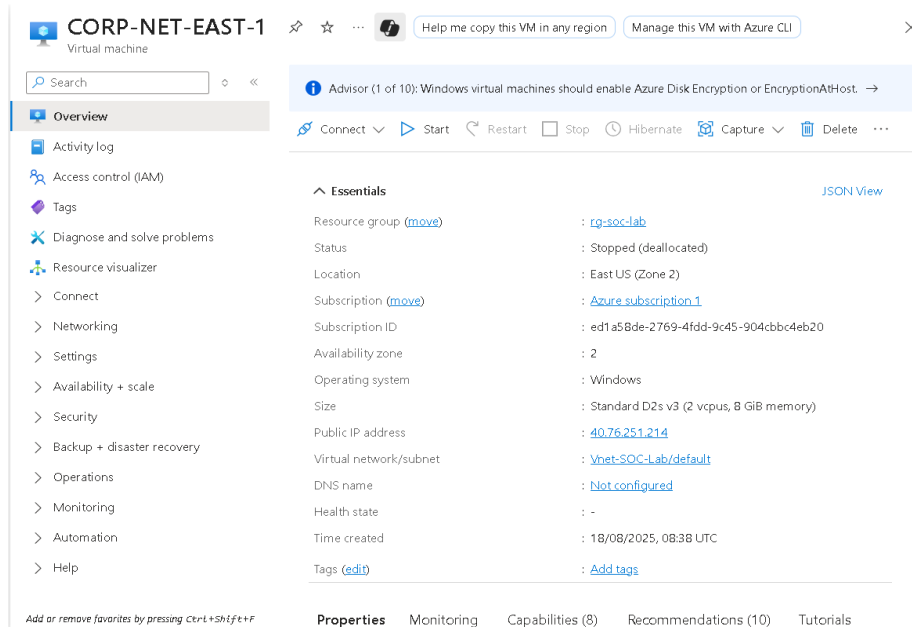


Figure 5. CORP-NET-EAST-1 overview showing size, public IP, and VNet.

5. VM access and management

The VM was reached over RDP from both Windows and Mac clients. The Windows Firewall inside the VM was turned off to widen the exposure further, and a ping to the public IP confirmed the host stayed reachable after the change.

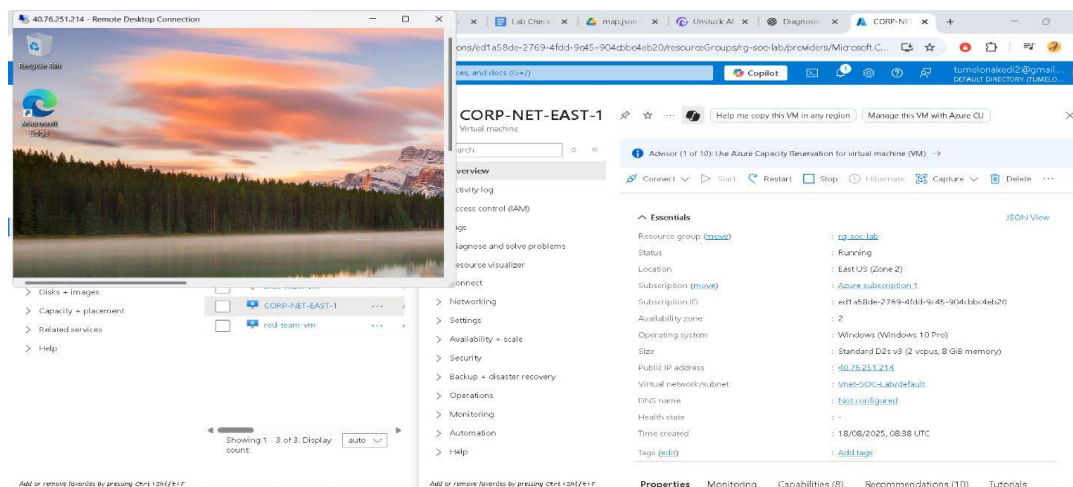


Figure 6. RDP session into CORP-NET-EAST-1 alongside the Azure portal.

Monitoring and logging

6. Local logging

Every login attempt against the VM writes to the Windows Event Viewer under Security logs. A failed sign-in is recorded as Event ID 4625. These events can be read on the host itself or shipped to Azure so they can be searched in one place.

7. Forwarding logs to Azure Sentinel

To pull the logs into a SIEM, the pipeline was wired up as follows:

- A Log Analytics workspace was created to store the collected logs.
- The workspace was connected to Microsoft Sentinel for detection and analysis.
- The Azure Monitor agent was installed on the VM and pointed at the workspace so events flow into Sentinel.

Attack simulation and analysis

8. Simulation

With the inbound NSG rules opened wide, the VM was left exposed to the internet. Within a short time, external hosts began hammering it with login attempts. Each failed attempt was logged as Event ID 4625 and forwarded to Sentinel for analysis.

Priority	Name	Port	Protocol	Source	Destination	Action
100	DANGER-AllowAnyCu...	Any	Any	Any	Any	Allow
65000	AllowVnetInBound	Any	Any	VirtualNetwork	VirtualNetwork	Allow
65001	AllowAzureLoadBalancerIn...	Any	Any	AzureLoadBalancer	Any	Allow
65500	DenyAllInBound	Any	Any	Any	Any	Deny
65000	AllowVnetOutBound	Any	Any	VirtualNetwork	VirtualNetwork	Allow
65001	AllowInternetOutBound	Any	Any	Any	Internet	Allow
65500	DenyAllOutBound	Any	Any	Any	Any	Deny

Figure 7. NSG rules for CORP-NET-EAST-1, including the deliberately permissive inbound rule.

9. Data visualization and querying

In Log Analytics, a KQL query pulls the failed logins and adds the attacker's location using the GeoIP watchlist. In short, the query:

- filters the SecurityEvent table for Event ID 4625 (failed logons),

- joins each event against the GeoIP watchlist to attach a city, country, and coordinates,
- counts the failures per source IP and location, and outputs the count, IP, coordinates, city, and country.

The result feeds a map workbook in Sentinel, where each point sits on the attacker's location and the heatmap color shows how many failed attempts came from that area.

Open a Sentinel workbook, go to Query settings, and run:

```
let GeoIPDB_FULL = _GetWatchlist("geoip");
let WindowsEvents = SecurityEvent;
WindowsEvents
| where EventID == 4625
| order by TimeGenerated desc
| evaluate ipv4_lookup(GeoIPDB_FULL, IPAddress, network)
| summarize FailureCount = count() by
    IPAddress, latitude, longitude, cityname, countryname
| project
    FailureCount,
    AttackerIp = IPAddress,
    latitude, longitude,
    city = cityname,
    country = countryname,
    friendly_location = strcat(cityname, " (", countryname, ")")
```

This loads the GeoIP watchlist and the SecurityEvent table, keeps only failed logons, sorts them newest first, attaches location data by IP, and tallies failures per IP and place. The final columns are the failure count, attacker IP, coordinates, city, country, and a combined city-and-country label.

To render it as a map, open Advanced settings in the workbook and paste the JSON below:

```
{
  "type": 3,
  "content": {
    "version": "KqlItem/1.0",
    "query": "let GeoIPDB_FULL = _GetWatchlist(\"geoip\");\nlet\nWindowsEvents = SecurityEvent;\nWindowsEvents | where EventID ==\n4625\n| order by TimeGenerated desc\n| evaluate\nipv4_lookup(GeoIPDB_FULL, IPAddress, network)\n| summarize\nFailureCount = count() by IPAddress, latitude, longitude,\ncityname, countryname\n| project FailureCount, AttackerIp =\nIPAddress, latitude, longitude, city = cityname, country =\ncountryname,\nfriendly_location = strcat(cityname, \" (\",\ncountryname, \")\");",
    "size": 3,
    "timeContext": { "durationMs": 2592000000 },
    "queryType": 0,
    "resourceType": "microsoft.operationalinsights/workspaces",
    "visualization": "map",
    "mapSettings": {
      "locInfo": "LatLong",
      "locInfoColumn": "countryname",
      "latitude": "latitude",
      "longitude": "longitude",
      "sizeSettings": "FailureCount",
      "sizeAggregation": "Sum",
      "opacity": 0.8,
    }
  }
}
```

```
"labelSettings": "friendly_location",
"legendMetric": "FailureCount",
"legendAggregation": "Sum",
"itemColorSettings": {
  "nodeColorField": "FailureCount",
  "colorAggregation": "Sum",
  "type": "heatmap",
  "heatmapPalette": "greenRed"
}
},
"name": "query - 0"
}
```

Visualization

The map workbook plots each source of failed logins, with point size and color scaled to the number of attempts. The heatmap makes the busiest origins obvious at a glance.

AttackerLocation

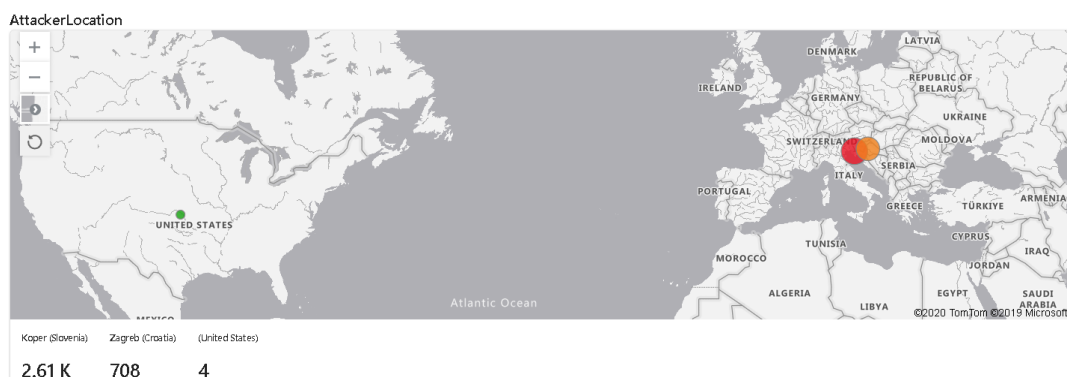


Figure 8. AttackerLocation workbook. Koper (Slovenia) led with about 2,610 failed attempts, Zagreb (Croatia) with 708, and the United States with 4.

Over the capture window, Koper in Slovenia generated roughly 2,610 failed sign-ins, Zagreb in Croatia 708, and a single source in the United States 4. The bulk of the pressure came from a small number of European hosts rather than a spread of locations.

Conclusion

The honeypot did what it was built to do. By leaving a Windows VM exposed and routing its security logs into Sentinel, the lab captured thousands of real failed login attempts and traced them back to their source locations. Pairing Azure Monitor, Log Analytics, and Sentinel turned raw event logs into a single queryable view and a map that shows where the attacks came from.

The exercise is a working example of how a small SOC setup can collect attacker activity, enrich it, and surface it for analysis, all within Azure and at almost no cost.